NAAN MUDHALVAN

PROJECT REPORT

AI WEATHERWISE

Deliver Intelligent Weather Forecasts and Insights

Submitted in partial fulfillment of the requirements for the Naan Mudhalvan Course

Academic Year: 2026–2027

S.No	Team	Name	Email id
1.	Team Leader	Kaviya.M	kaviyakaviya5447@gmail.com
2.	Team Member	Kamalini.T	kamalini654@gmail.com
3.	Team Member	Bhuvana.P	pooji0416@gmail.com
4.	Team Member	Deepikasree.V	dd7019600@gmail.com
5.	Team Member	Durga.M	sankarisankari5418@gmail.com

TABLE OF CONTENTS

1. Introduction
2. Objectives
3. Existing System
4. Proposed System
5. System Requirements
6. System Design
7. Modules
8. Implementation
9. Testing
10. Advantages
11. Limitations
12. Future Enhancements
13. Conclusion 14. References

1. INTRODUCTION

- Weather forecasting is the process of predicting atmospheric conditions for a particular place and time. Traditional weather information can be difficult for users to interpret because it may contain large amounts of data.
- AI WeatherWise aims to present important weather information in a simple interface. The system can use available weather data and intelligent processing to provide forecast information and useful insights. This helps users understand current and upcoming weather conditions more easily.

2. OBJECTIVES

- To provide weather information in a simple and user-friendly format.
- To display important weather parameters such as temperature, humidity, wind and rainfall probability.
- To provide forecast information for upcoming periods.
- To use intelligent analysis to generate understandable weather insights.
- To help users plan daily activities based on weather conditions.

3. EXISTING SYSTEM

- In a basic weather information system, users mainly view raw weather values or simple forecast details. Such systems may not always provide easy-to-understand insights. Users may need to interpret several weather parameters themselves.
- Limitations of the existing approach include complex information presentation, limited personalized insights, and difficulty in understanding multiple weather parameters together.

4. PROPOSED SYSTEM

- AI WeatherWise proposes a simple weather application that collects or receives weather data, processes the information, and presents meaningful forecast details to the user.
- The proposed system focuses on clear presentation, forecast information, and intelligent insights. It can help users understand whether conditions are suitable for common outdoor activities and make better-informed plans.

5. SYSTEM REQUIREMENTS

5.1 Hardware Requirements

• Computer/Laptop or Smartphone

• Minimum 4 GB RAM recommended

• Internet connection

• Standard display and input devices

5.2 Software Requirements

• Operating System: Windows / Android or other supported OS

• Web browser or supported application environment

• Programming environment as used for implementation

• Weather API or weather data source

• Internet connection for live weather data

6. SYSTEM DESIGN

6.1 System Architecture

User → Weather Interface → Weather Data Source/API → Data Processing → AI/Insight Analysis → Forecast & Insights → User

6.2 Basic Data Flow

• User enters/selects a location.

• The system requests weather data.

• Weather data is received and processed.

• Important parameters are extracted.

• The intelligent analysis generates useful insights.

• The forecast and insights are displayed to the user.

6.3 Use Case Description

The main user can select a location, view current weather, view forecast information, and read weather insights. The system handles weather-data retrieval, processing, and presentation.

7. MODULES

7.1 User Interface

Provides a simple interface for selecting a location and viewing weather information.

7.2 Weather Data Collection

Obtains weather information from the selected weather data source or API.

7.3 Weather Data Processing

Processes temperature, humidity, wind, rainfall and other available parameters.

7.4 AI Weather Analysis

Analyses the available weather information and converts it into understandable insights.

7.5 Forecast Display

Displays current conditions and upcoming forecast information in a clear format.

8. IMPLEMENTATION

- The implementation consists of a user interface, weather-data retrieval component, data processing logic, and an insight-generation component. The user first selects a location. The application then retrieves the available weather information and processes the data.
- The processed information is displayed using clear labels and forecast sections. Intelligent rules or AI-based processing can be used to generate simple insights from the available weather parameters.

9. TESTING

Test Case	Input	Expected Result	Status
Location Search	Valid location	Weather information displayed	Pass
Weather Data	Available API/data	Current weather displayed	Pass
Forecast	Selected location	Forecast details displayed	Pass

Invalid Location	Invalid/empty input	Suitable error/validation shown	Pass
Insight Display	Weather parameters	Readable weather insight displayed	Pass

10. ADVANTAGES

• Simple and user-friendly weather information.

• Provides multiple weather parameters in one place.

• Forecast information supports daily planning.

• Intelligent insights make weather data easier to understand.

• Can be extended with additional weather-related features.

11. LIMITATIONS

• Forecast quality depends on the accuracy and availability of the weather data source.

• Internet connectivity may be required for live data.

• AI-generated insights depend on the data supplied to the system.

• Extreme weather conditions may require information from official authorities.

12. FUTURE ENHANCEMENTS

• Add severe weather alerts and notifications.

• Add longer forecast periods where supported by the data source.

• Provide personalized activity suggestions based on weather conditions.

• Add multilingual support including Tamil.

• Add historical weather analysis and charts.

• Improve intelligent insights using advanced AI/ML techniques.

Core System Script Implementations

1. Application Entry Point (index.js)

The **index.js** file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
    });
  })
  .catch((error) => {
    console.error('Database connection failed. Express server aborted:', error.message);
    process.exit(1);
  });
```

13. Database Configuration

1. Configure MongoDB:

- Install Mongoose.
  In your Node.js project directory, run:

npm install mongoose • Create database connection.

- Create Schemas & Models.

Create Database connection:

- Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions Location

Schema:

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

User schema:

Db connection:

- This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

14 Project Executions:

- The AI weatherwise backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client.

Running the Backend Server

Install all required dependencies:

```
npm install
```

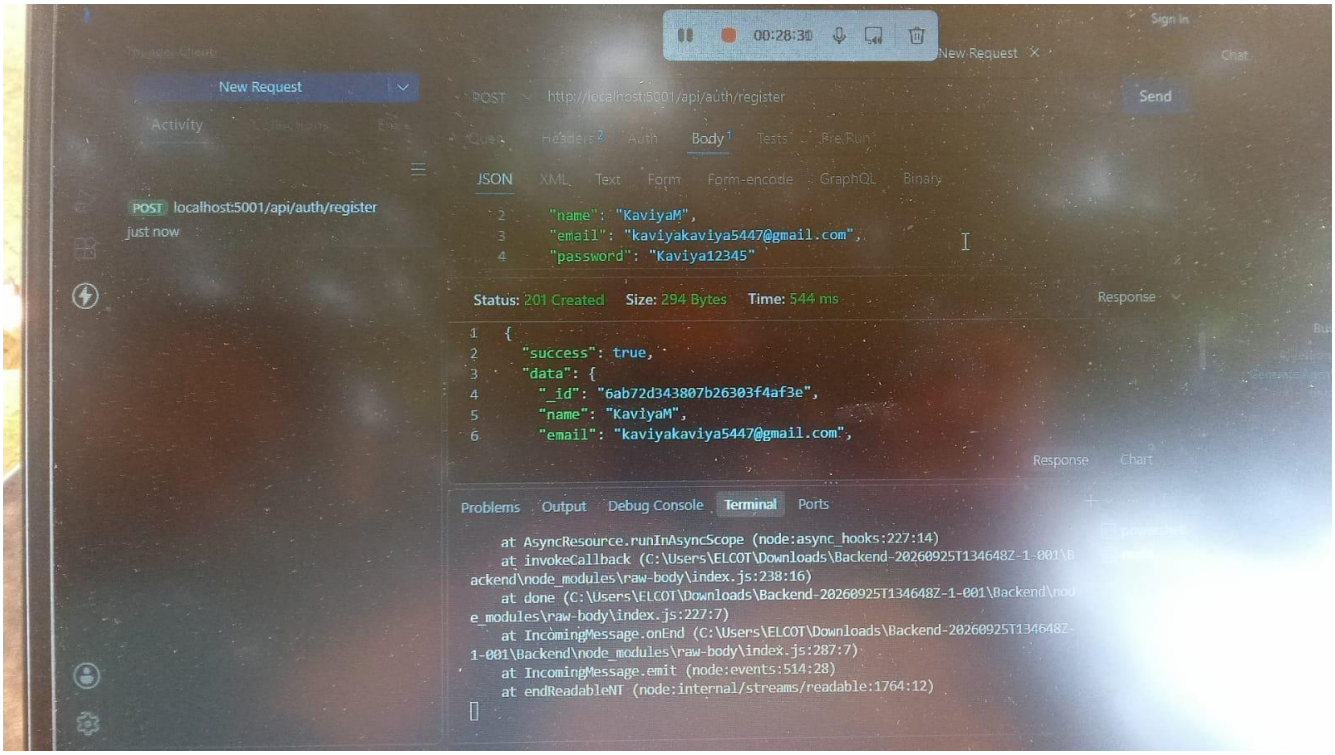
Start the development server:

```
npm start
```

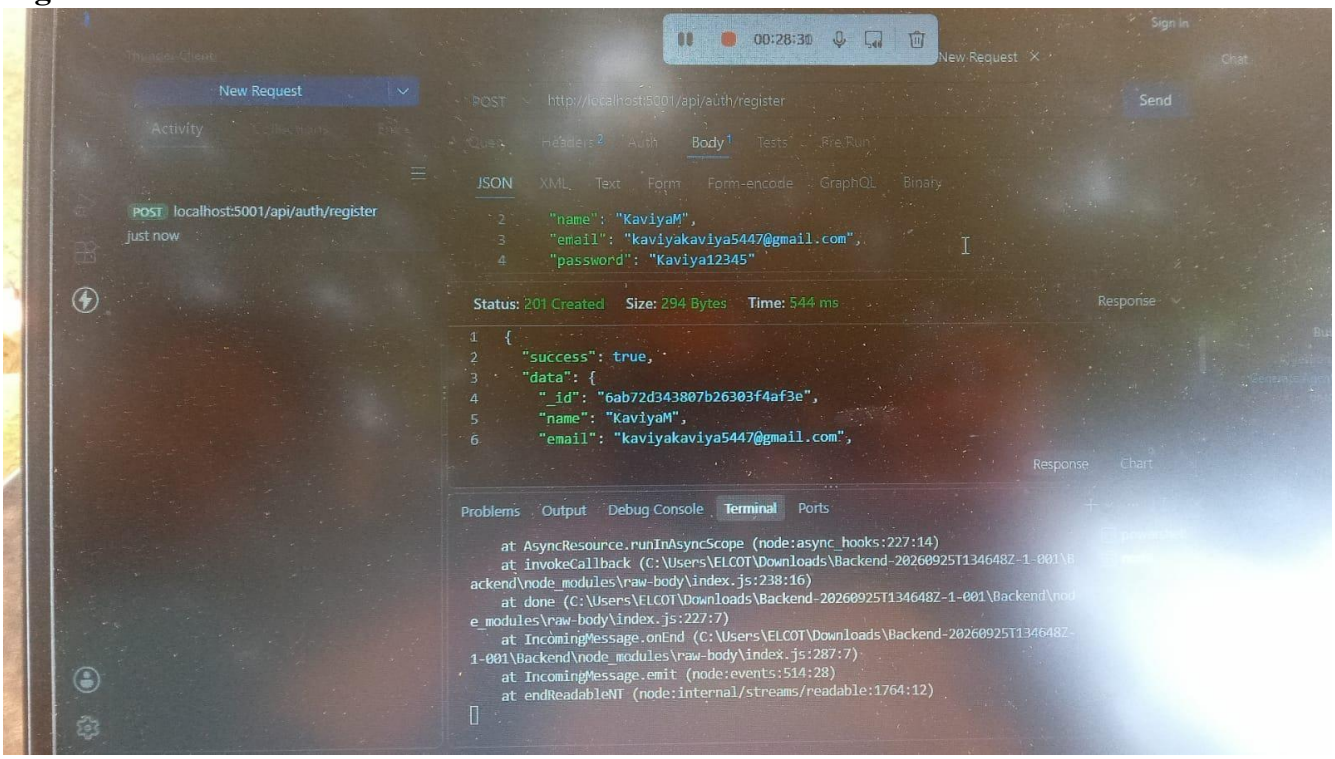
After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

API testing:

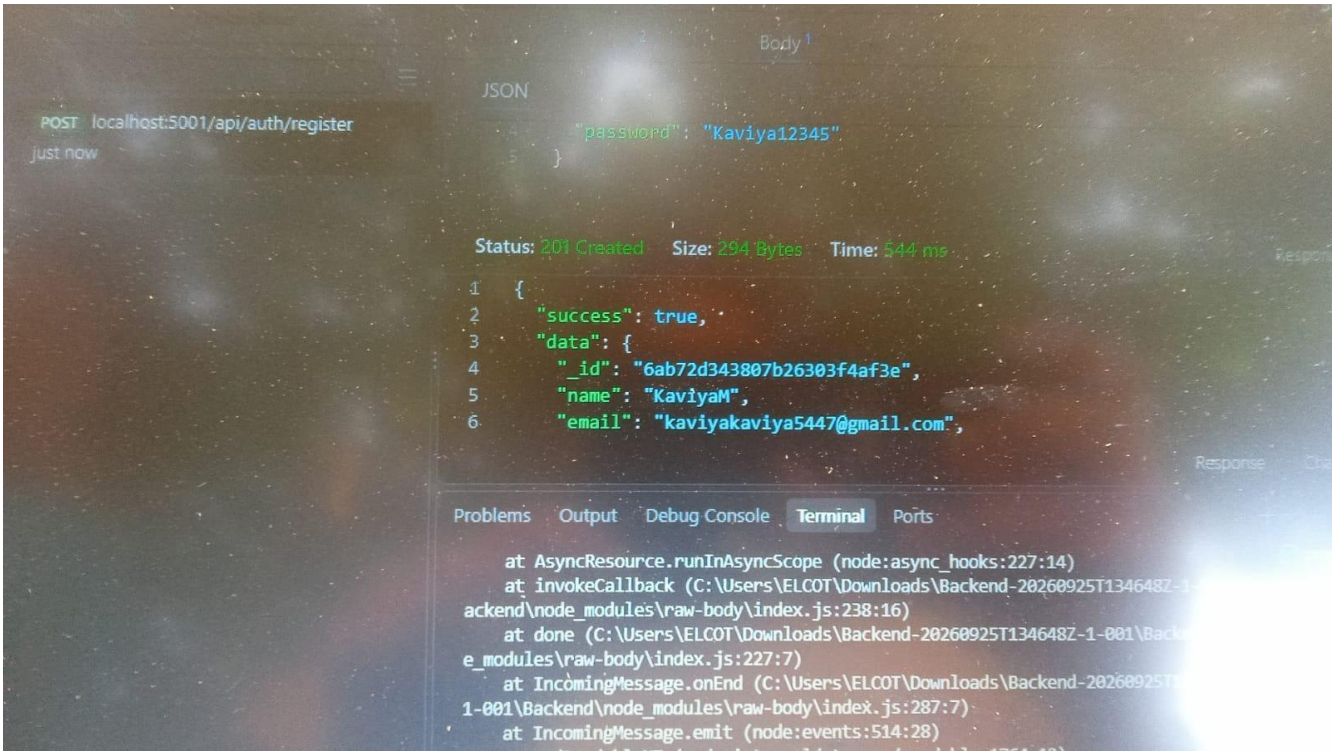
Register API:



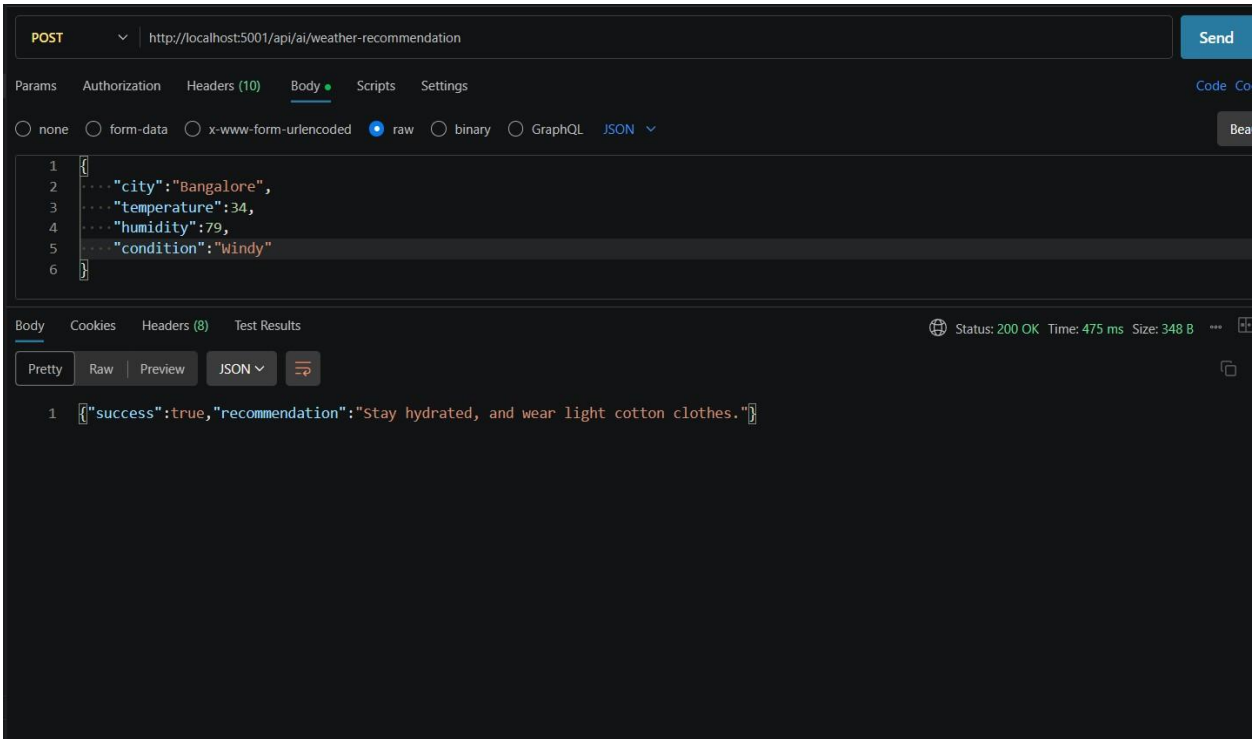
Login API:



Weather API:



Recommendation API:



15. CONCLUSION

- AI WeatherWise is designed to make weather information simple, accessible, and useful. The project combines weather data, processing, and intelligent insights to help users understand current and future weather conditions.

- The system provides a foundation that can be enhanced with alerts, multilingual support, historical analysis, and advanced AI techniques. Thus, AI WeatherWise demonstrates how intelligent technologies can be applied to everyday weather information.

16. REFERENCES

- Official documentation of the selected weather API/data source.
- Python / JavaScript / web development documentation as applicable to the implementation.
- Naan Mudhalvan course learning materials.
- General reference materials on weather forecasting and artificial intelligence.